Spring Tool Suit
The newest release STS 2.0 ehhances the toolset's ease of use, so developers to get more done, quickly.

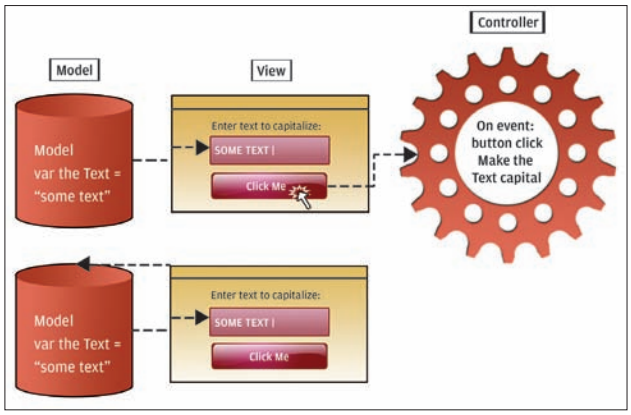
Nokia's Qt software division
Released Qt version 4.5, that will expand cross-platform capabilities and increase performance.

views such as a wizard view or a simple form view. The view is not dependent on the control; however, it needs to be aware of the model to display the data in it.

The Controller

The controller represents the main logic of your program. It is like the brain which manages the other elements of the MVC program. It accesses and modifies the data of the model in response to the user's interactions with the view. It is again not concerned with what the view looks like. It is an action-reaction system, where certain interactions with the view will result in modifications in the model, or changes in the view.

As in the address book example, the controller would then



be responsible for adding / removing / modifying contact entries, sorting the data in response to triggers from the view, performing searches, etc.

The control is again not dependent on the view, and only responds to events generated by it. However, it needs to be aware of the model in order to make changes to it in response to the events.

The result

In most cases, the end user will not care about the internal mechanism by which the program chooses to work. Unless a structure is clearly defined, it can become very difficult for a team to collaborate on the coding. Using an MVC design does not eliminate this problem. However, it does make it easier to avoid.

In a project built with MVC principles, you can have a professional database developer working away at adding more kinds of data formats that can be handled to the model, while a designer works away on adding more views and making the views better, while more people are working on the core functionality and improving the performance of the controller. Each can work separately and independently of each other, by following simple guidelines as to how the three would interact.

Since the separate elements are decoupled from each other, it also means that there is no model-specific code in the controller, and no controller specific content in the view. This means that they can easily be 'unplugged' from one program and put into another, with little adaptation required. This makes the code very reusable.

An Example

Here is simple explanation of how things go in a simple MVC program. We take the example of a simple program, which has a textbox and a button. When you click on the button, the content of the textbox is capitalised. Here, let's enumerate the role of the model, the view and the controller.

- The model stores the contents of the textbox, and provides a way of accessing and modifying it. It also registers an event when this data is change.
- The view creates the visual part of the application, the button and the textbox, and the window in which they reside. It also registers an event when the button is clicked.
- The controller responds to any events generated by the view (or elsewhere), and modify the contents of the model. So when the user clicks on the button, it responds to that event, and capitalises the textbox content stored in the model.

The same view can be used for a program which would encrypt and decrypt the contents of the text box, and the same controller can be used in a word processing program for providing capitalisation as a function.

Some kind of mechanism for updating the view in response to changes in the model should also be in place and this is where a framework comes in. Although it is possible to make it entirely yourself, why not let the framework do the grunt work for you?

MVC Frameworks

MVC is not something you impose on your programs without a reason. For very small projects, or one time projects, it doesn't make much sense to adopt an MVC design. That doesn't mean you abandon any kind of pattern, just that this might not be suitable for the job. You can adapt the design to your own needs, for example combine the model and controller. Here are some examples:

- **Java Swing** : A toolkit for Java, this supports more than one MVC design, both at the application level and the component level.
- **Qt** : A very popular toolkit on which KDE, a major desktop environment for Linux, is built.
- **PureMVC**: A light–weight framework, which only imposes the design pattern, leaving the rest of the implementation up to the developer. Originally developed for Flash / Flex using ActionScript, however now available for many other languages such as Java, JavaScript, C#, PHP and Python.
- **GTK+**: Another toolkit made for Linux, originally for the GIMP, also the toolkit on which Gnome is built (Gnome is another major Linux desktop environment).
- **Cocoa**: The API for building applications for MacOS X
- Cairngorm: An MVC framework by Adobe for their Flex SDK.

MVC - The Perfect Solution

MVC is by no means a perfect solution to all kinds of problems that come up during development. In many cases, adopting such a framework can be harmful.

Deciding what pattern to use depends hugely on what you are trying to accomplish and how. It is not that easy to decide whether to use MVC or not – it is a matter of experience. Although one can assume scenarios where using such a design may be appropriate and where it may not, it is difficult to create strict rules. Generally, small projects or those developed individually need not require such an approach.

MVC Alternatives

Over time people with experience in software development see common traits, common problems that any project is bound to face irrespective of platform, language or purpose. It is using that experience that people have come up with many interesting ways to counter such problems, and MVC is only one of them. Here are some promising ones:

- **Model-View-Adapter**: it is similar to MVC, but additionally allows the same view to be used with multiple models instead of the other way around.
- **Blackboard system**: here the software is segregated by specialisation. It is similar to a group of specialists working on a problem on a blackboard. In a programming sense, different parts of the program are specialised for different tasks, and each iteratively updates a 'blackboard' containing the problem, till a solution is obtained.
- **IoC**: IoC stands for inversion of control. It is a principle of programming in which we invert the sequence of control. Instead of defining what part of your code will run what library function, you instead define what part of the library requires your code to run.

These are just some interesting examples of how one can think about organising their functionality. What you use should depend on what gives you the greatest flexibility and room for expansion in the domain of your project. ■

kshitij.sobti@thinkdigit.com